

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования

**«Национальный исследовательский Нижегородский
государственный университет им. Н.И. Лобачевского»
(ННГУ)**

Институт информационных технологий, математики и механики

Центр прикладных информационных технологий

Направление подготовки: «Фундаментальная информатика и
информационные технологии»

Профиль подготовки: «Инженерия программного обеспечения»

Отчет по практической работе
на тему:

«Алгоритм Дейкстры на 3-куче и 15-куче»

Выполнил:

студент группы 3822Б1ФИ1

_____ Чистов А. Д.

Проверил:

_____ Уткин Г. В.

Нижний Новгород
2024 г.

Содержание

Введение	2
1 Основные понятия теории графов	3
Заключение	4
Список литературы	5

Введение

Одной из центральных задач теории графов является нахождение кратчайших путей в графах, что имеет широкое практическое применение в задачах маршрутизации, логистики, сетевого планирования и моделирования. Алгоритм Дейкстры, предложенный Эдсгером Дейкстрой в 1959 году, стал классическим подходом для решения этой задачи на графах с ненулевыми положительными весами рёбер [1]. Алгоритм Дейкстры позволяет находить кратчайший путь от одной вершины графа до всех остальных, и основой его эффективности является использование структур данных, таких как куча, для оптимизации извлечения минимального значения.

Обычно в алгоритме Дейкстры используется двоичная куча, обеспечивающая хорошую производительность для стандартных задач. Однако для улучшения производительности при работе с большими графами и для различных практических применений может использоваться D-арная куча (например, 3-куча, 15-куча и др.), позволяющая сократить количество операций по сравнению с двоичной кучей за счёт уменьшения высоты дерева. В частности, 3-куча и 15-куча могут быть полезны для ускорения алгоритма в зависимости от структуры графа и особенностей доступа к памяти.

Цели и задачи данной работы:

1. Исследовать тематическую литературу.
2. Реализовать класс DHeap.
3. Реализовать алгоритм Дейкстры.
4. Провести экспериментальное сравнение производительности и вычислительной эффективности алгоритма на различных типах графов.
5. Сделать выводы о проделанной работе.

1 Основные понятия теории графов

Заключение

В ходе проведенного исследования был рассмотрен алгоритм Дейкстры на d -куче, с фокусом на реализацию с параметрами $d = 3$ и $d = 15$. Анализ полученных результатов, а также построенные графики, показали, что при увеличении параметра d существенного различия в производительности алгоритма не наблюдается. Незначительные отклонения в результатах можно объяснить погрешностями, связанными с особенностями работы операционной системы Windows, что потенциально влияет на время выполнения и управляет распределением ресурсов во время исполнения алгоритма.

Во время исследования были достигнуты следующие цели и выполнены поставленные задачи:

1. Изучены особенности алгоритма Дейкстры и его модификации для d -куч.
2. Реализованы и протестированы алгоритмы на d -кучах с параметрами $d = 3$ и $d = 15$.
3. Проведен сравнительный анализ времени выполнения алгоритмов и построены графики зависимости производительности от параметров.

Полученные результаты позволяют заключить, что увеличение значения параметра d в d -куче для реализации алгоритма Дейкстры не оказывает значительного влияния на эффективность. Это объясняется тем, что при больших значениях d по-прежнему доминирует природа самого алгоритма поиска кратчайших путей, где основное время выполнения тратится на операции извлечения и обновления приоритетов.

В заключение, можно отметить, что практическое применение d -кучи с большими значениями d может оказаться нецелесообразным, так как увеличивается сложность структуры данных без значительных улучшений в скорости работы алгоритма. Тем не менее, данное исследование позволило углубить понимание свойств d -куч в контексте алгоритма Дейкстры и расширить знания о том, как особенности архитектуры системы могут оказывать влияние на результаты вычислений.

Список литературы

1. Алгоритм Дейкстры // Википедия: свободная энциклопедия.
URL: https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm.
2. Кормен Т.Х., Лейзерсон Ч.Е., Ривест Р.Л., Штайн К. Алгоритмы: Построение и анализ / Т.Х. Кормен, Ч.Е. Лейзерсон, Р.Л. Ривест, К. Штайн. — 3-е изд. — М.: Вильямс, 2011.
3. Скиен С. Алгоритмы. Руководство по разработке / С. Скиен. — М.: Вильямс, 2006. — 800 с.: ил. ISBN 978-5-8459-0805-7.

Приложение А. Реализация DHeap

Листинг 1: Приложение А. Реализация DHeap

```
1 class DHeap {
2 public:
3     DHeap(int d);
4     void Insert(int key, int value);
5     std::pair<int, int> ExtractMin();
6     bool IsEmpty() const;
7
8 private:
9     size_t MinChild(size_t i);
10    size_t LastChild(size_t i) const;
11    size_t FirstChild(size_t i) const;
12    size_t Father(size_t i) const;
13    void SiftDown(size_t i);
14    void SiftUp(size_t i);
15
16    int d_;
17    std::vector<std::pair<int, int>> v_;
18 };
19
20 size_t DHeap::MinChild(size_t i) {
21     size_t first = FirstChild(i);
22     if (first == -1) return i;
23
24     size_t last = LastChild(i);
25     size_t min_idx = first;
26
27     for (size_t j = first + 1; j <= last; ++j) {
28         if (v_[j].second < v_[min_idx].second) {
29             min_idx = j;
30         }
31     }
32     return min_idx;
33 }
34
35 size_t DHeap::LastChild(size_t i) const {
36     size_t first = FirstChild(i);
37     return (first == -1) ? -1 : std::min(first + d_ - 1, v_.size() - 1);
38 }
39
40 size_t DHeap::FirstChild(size_t i) const {
41     size_t k = d_ * i + 1;
42     return (k < v_.size()) ? k : -1;
43 }
```

```

44
45 size_t DHeap::Father(size_t i) const {
46     return (i - 1) / d_;
47 }
48
49 void DHeap::SiftDown(size_t i) {
50     int key0 = v_[i].first;
51     int value0 = v_[i].second;
52     size_t c = MinChild(i);
53
54     while (c != i && value0 > v_[c].second) {
55         v_[i] = v_[c];
56         i = c;
57         c = MinChild(i);
58     }
59     v_[i] = { key0, value0 };
60 }
61
62 void DHeap::SiftUp(size_t i) {
63     int key0 = v_[i].first;
64     int value0 = v_[i].second;
65     size_t p = Father(i);
66
67     while (i != 0 && v_[p].second > value0) {
68         v_[i] = v_[p];
69         i = p;
70         p = Father(i);
71     }
72     v_[i] = { key0, value0 };
73 }
74
75 DHeap::DHeap(int d) : d_(d) {
76     if (d_ < 2) throw std::runtime_error("Degree of heap must be at least 2.");
77 }
78
79 void DHeap::Insert(int key, int value) {
80     v_.push_back({ key, value });
81     SiftUp(v_.size() - 1);
82 }
83
84 std::pair<int, int> DHeap::ExtractMin() {
85     if (v_.empty()) throw std::runtime_error("Heap is empty");
86
87     std::pair<int, int> min_element(v_[0]);
88     v_[0] = v_.back();
89     v_.pop_back();

```

```

90     if (!v_.empty()) SiftDown(0);
91
92     return min_element;
93 }
94
95 bool DHeap::IsEmpty() const {
96     return v_.empty();
97 }

```

Приложение Б. Реализация Дейкстры

Листинг 2: Приложение Б. Реализация Дейкстры

```

1  vector<vector<pair<int, int>>> generateGraph(int n, int m, int q, int r) {
2      m = min(n * (n - 1) / 2, m);
3      vector<vector<pair<int, int>>> adj_list(n);
4
5      std::random_device rd;
6      std::mt19937 gen(rd());
7      std::uniform_int_distribution<> dist(q, r);
8
9      vector<vector<bool>> was(n, vector<bool>(n, false));
10     int edge_count = 0;
11
12     while (edge_count < m) {
13         int i = rand() % n;
14         int j = rand() % n;
15         if (i != j && !was[i][j]) {
16             int weight = dist(gen);
17
18             adj_list[i].push_back({ j, weight });
19             adj_list[j].push_back({ i, weight });
20
21             was[i][j] = was[j][i] = true;
22             ++edge_count;
23         }
24     }
25     return adj_list;
26 }
27
28 double dijkstra(vector<int>& dist, vector<int>& up, vector<vector<pair<int,
29     int>>> graph, int n, int d, int start) {
30     vector<bool> processed(n, false);
31     vector<int> index(n);
32     vector<int> name(n);

```

```

33     for (int i = 0; i < n; ++i) {
34         up[i] = -1;
35         dist[i] = INT_MAX;
36         index[i] = i;
37         name[i] = i;
38     }
39
40     dist[start] = 0;
41     up[start] = start;
42     DHeap dHeap(d);
43     dHeap.Insert(start, dist[start]);
44
45     auto begin = std::chrono::high_resolution_clock::now();
46
47     while (!dHeap.IsEmpty()) {
48         pair<int, int> minPair = dHeap.ExtractMin();
49         int i = minPair.first;
50         int currentDist = minPair.second;
51
52         if (processed[i]) continue;
53         processed[i] = true;
54
55         vector<pair<int, int>> edge = graph[i];
56         for (int k = 0; k < edge.size(); k++) {
57             int j = edge[k].first;
58             int newDist = edge[k].second + currentDist;
59
60             if (newDist < dist[j]) {
61                 dist[j] = newDist;
62                 up[j] = i;
63                 dHeap.Insert(j, dist[j]);
64             }
65         }
66     }
67
68     auto end = std::chrono::high_resolution_clock::now();
69     std::chrono::duration<double> executionTime = end - begin;
70     std::cout << "Execution time: " << executionTime.count() << " seconds\n"
71     ;
72     return executionTime.count();
73 }

```